

MANUEL DE DIAGNOSTIC DES DESIGN PATTERNS GoF

Reconnaître le bon pattern à partir des symptômes réels du code

Idée directrice : on ne choisit pas un pattern parce qu'on connaît son nom. On le reconnaît parce qu'une situation de conception produit des symptômes récurrents.

- 23 patterns du Gang of Four
- Approche par symptômes situationnels
- Exemples principalement en Java
- Avant / après, pièges, règles mentales
- Carte de diagnostic rapide en fin de manuel

Version pédagogique — 2026

Comment utiliser ce manuel

Les patterns GoF ne sont pas des recettes obligatoires. Ils servent à nommer des solutions récurrentes à des tensions de conception récurrentes. L'objectif de ce manuel est donc de partir du terrain : qu'est-ce que le développeur voit dans son code, dans ses dépendances, dans ses évolutions ou dans ses tests qui doit lui faire penser à un pattern précis ?

La question à se poser n'est pas « Quel pattern puis-je mettre ici ? » mais « Quel problème de variation, de couplage, de construction ou de collaboration est en train d'apparaître ? »

Les trois familles GoF

Ce que vous observez	Ce que cela signifie	Réflexe
Création	Le problème est surtout : qui crée quoi, comment, et avec quelles variantes ?	Factory Method, Abstract Factory, Builder, Prototype, Singleton
Structure	Le problème est surtout : comment connecter, composer, envelopper ou simplifier des objets ?	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
Comportement	Le problème est surtout : qui décide, qui réagit, comment les objets collaborent-ils ?	Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

Méthode de lecture en 6 questions

- Qu'est-ce qui varie ? Le type d'objet, un algorithme, un état, une famille de composants, une séquence d'étapes ?
- Où sont les `if`, `switch`, `instanceof`, `new` et dépendances concrètes qui se répètent ?
- Quelle partie du code change le plus souvent quand une nouvelle variante apparaît ?
- Le problème vient-il de la création, de la structure ou de la collaboration ?
- Le pattern réduit-il réellement le coût du prochain changement ou ajoute-t-il seulement de l'abstraction ?
- Le code est-il déjà assez complexe pour justifier ce pattern ?

Sommaire des 23 patterns

Création — Factory Method, Abstract Factory, Builder, Prototype, Singleton

Structure — Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

Comportement — Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

Patterns de création

Le symptôme commun est une création d'objets qui devient une responsabilité à part entière : choix de type, familles compatibles, construction complexe, clonage ou contrôle du cycle de vie.

Dans cette partie, cherchez moins la forme UML parfaite que le signal concret indiquant que la responsabilité actuelle n'est plus placée au bon endroit.

01. Factory Method

CRÉATION

Essence — Laisser une classe ou une sous-classe décider quelle implémentation concrète doit être instanciée, sans faire porter ce choix au code métier.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous voyez des `new` différents choisis par `if` / `switch` dans un traitement qui devrait surtout exécuter une règle métier.
- Une classe sait trop de choses sur les classes concrètes qu'elle instancie.
- Une même opération générale existe, mais l'objet exact à créer dépend du contexte, d'une sous-classe ou d'un type de traitement.
- Ajouter une nouvelle variante vous oblige à modifier une classe centrale simplement pour ajouter un nouveau `new`.
- Vous voulez tester la logique générale sans figer l'objet concret créé.
- Le workflow reste identique mais le produit créé varie : document PDF/CSV, notification mail/SMS, transport camion/bateau, etc.

2. Le symptôme central

À retenir : Quand le `new` contient une décision polymorphe, demandez-vous si cette décision ne devrait pas être déléguée.

3. Scénario concret

Un exportateur exécute toujours le même workflow (préparer, exporter, journaliser), mais le type de document créé dépend du sous-type d'exportateur.

4. À quoi ressemble le code avant ?

```
if (format.equals("PDF")) {
    document = new PdfDocument();
} else if (format.equals("CSV")) {
    document = new CsvDocument();
}
document.generate();
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
abstract class ExportService {
    protected abstract Document createDocument();

    public void export() {
        Document doc = createDocument();
        doc.generate();
    }
}
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- L'objet est trivial et ne varie jamais.

- Vous n'avez qu'un simple besoin de configuration d'objet : un constructeur ou un Builder suffit.
- Le choix des variantes appartient à la configuration du conteneur d'injection de dépendances et n'a pas besoin d'être codé dans une hiérarchie.

7. Question de diagnostic

Le code qui exécute le traitement devrait-il vraiment connaître la classe concrète qu'il crée ?

8. Ne pas confondre

Simple Factory centralise un choix ; Factory Method fait du point de création une méthode redéfinissable ; Abstract Factory crée des familles d'objets compatibles.

9. Règle mentale en une ligne

Quand le `new` contient une décision polymorphe, demandez-vous si cette décision ne devrait pas être déléguée.

02. Abstract Factory

CRÉATION

Essence — Créer des familles cohérentes d'objets liés sans exposer leurs classes concrètes.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous devez créer plusieurs objets qui doivent impérativement appartenir à la même variante ou au même environnement.
- Vous avez des couples ou familles comme `WindowsButton/WindowsMenu`, `AWSStorage/AWSQueue`, `MySqlRepository/MySqlDialect`.
- Le code contient plusieurs ``if (platform...)`` répétés pour fabriquer des objets différents mais corrélés.
- Changer d'environnement doit remplacer tout un ensemble de composants, pas seulement un objet.
- Vous craignez des combinaisons incohérentes : un bouton Mac avec un menu Windows, un driver PostgreSQL avec un dialecte MySQL.
- Les clients devraient manipuler des interfaces mais ne pas connaître le nom des implémentations.

2. Le symptôme central

À retenir : Si changer de "monde" impose de changer plusieurs objets ensemble, pensez famille de produits → Abstract Factory.

3. Scénario concret

Une application doit fonctionner en mode Cloud AWS ou Azure. Chaque mode fournit stockage, file de messages et service d'identité compatibles entre eux.

4. À quoi ressemble le code avant ?

```
if (cloud.equals("AWS")) storage = new S3Storage();
if (cloud.equals("AWS")) queue = new SqsQueue();
if (cloud.equals("AZURE")) storage = new BlobStorage();
if (cloud.equals("AZURE")) queue = new ServiceBusQueue();
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
interface CloudFactory {
    Storage createStorage();
    Queue createQueue();
}

CloudFactory factory = config.cloudFactory();
Storage storage = factory.createStorage();
Queue queue = factory.createQueue();
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Vous ne créez qu'un seul type de produit.
- Les produits n'ont aucune contrainte de compatibilité entre eux.
- La famille est stable et une simple configuration DI suffit sans logique de création particulière.

7. Question de diagnostic

Quand je change de plateforme/environnement, dois-je remplacer plusieurs objets cohérents ensemble ?

8. Ne pas confondre

Factory Method varie un produit ; Abstract Factory varie une famille ; Builder varie la construction d'un objet complexe.

9. Règle mentale en une ligne

Si changer de "monde" impose de changer plusieurs objets ensemble, pensez famille de produits → Abstract Factory.

03. Builder

CRÉATION

Essence — Construire progressivement un objet complexe en séparant le processus de construction de sa représentation finale.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Un constructeur possède beaucoup de paramètres, surtout plusieurs booléens, chaînes ou valeurs optionnelles.
- Vous ne savez plus ce que signifie `new Report(true, false, 20, null, ...)` à la lecture.
- Il existe de nombreuses combinaisons valides d'options et vous voulez les rendre lisibles.
- La création doit respecter un ordre ou plusieurs étapes : configurer, valider, assembler, finaliser.
- Vous avez une explosion de constructeurs surchargés (telescoping constructors).
- Vous voulez rendre l'objet final immuable tout en gardant une construction ergonomique.

2. Le symptôme central

À retenir : Quand le constructeur devient une phrase illisible, transformez-le en construction progressive lisible.

3. Scénario concret

Créer une requête HTTP avec URL obligatoire mais timeout, headers, authentification, retry et body optionnels.

4. À quoi ressemble le code avant ?

```
HttpRequest r = new HttpRequest(url, 3000, true, token, headers, null, 3, false);
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
HttpRequest r = HttpRequest.builder(url)
    .timeout(3000)
    .bearerToken(token)
    .headers(headers)
    .retries(3)
    .build();
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- L'objet a deux ou trois paramètres évidents.
- Vous cherchez seulement à choisir entre plusieurs sous-types : regardez Factory.
- Le véritable problème est une séquence algorithmique fixe et non la construction d'un objet.

7. Question de diagnostic

La difficulté vient-elle surtout du nombre d'options et de la lisibilité de la construction ?

8. Ne pas confondre

Builder = comment construire ; Factory = quoi créer ; Prototype = partir d'un exemplaire existant.

9. Règle mentale en une ligne

Quand le constructeur devient une phrase illisible, transformez-le en construction progressive lisible.

04. Prototype

CRÉATION

Essence — Créer un nouvel objet par copie d'un prototype existant plutôt qu'en reconstruisant tout depuis zéro.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Créer un objet est coûteux : parsing, accès disque, préparation d'un graphe, calcul initial important.
- Vous avez des objets presque identiques qui ne diffèrent que par quelques propriétés.
- Le type concret à créer n'est connu qu'à l'exécution mais vous disposez déjà d'un exemplaire configuré.
- Vous dupliquez de longues séquences de configuration avant de changer deux champs.
- Vous avez besoin de modèles ("templates") que l'utilisateur peut cloner puis personnaliser.
- La création classique dépend de nombreuses sous-structures déjà préparées.

2. Le symptôme central

À retenir : Quand vous dites "je veux le même, mais légèrement différent", pensez Prototype.

3. Scénario concret

Un éditeur graphique propose des formes préconfigurées. Dupliquer une forme doit conserver style, dimensions et métadonnées tout en produisant un nouvel objet.

4. À quoi ressemble le code avant ?

```
Rectangle copy = new Rectangle();
copy.setWidth(original.getWidth());
copy.setHeight(original.getHeight());
copy.setBorder(original.getBorder());
copy.setFill(original.getFill());
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
Shape copy = original.copy();
copy.setX(200);
copy.setY(100);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- La copie profonde est plus complexe et plus risquée que la reconstruction.
- Les objets contiennent des ressources non clonables (socket, thread, transaction active).
- La construction est simple et peu coûteuse.

7. Question de diagnostic

Ai-je déjà un exemplaire correctement configuré qu'il serait plus naturel de copier que de reconstruire ?

8. Ne pas confondre

Prototype clone un état existant ; Builder reconstruit étape par étape ; Flyweight partage au lieu de copier.

9. Règle mentale en une ligne

Quand vous dites “je veux le même, mais légèrement différent”, pensez **Prototype**.

05. Singleton

CRÉATION

Essence — Garantir une instance unique et fournir un point d'accès global contrôlé à cette instance.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Une ressource doit réellement avoir une seule instance logique dans le processus : registre, coordinateur, gestionnaire très spécifique.
- Plusieurs instances provoqueraient un conflit ou une incohérence globale.
- Vous avez besoin d'un contrôle explicite sur le cycle de vie de cette instance.
- Le code crée accidentellement plusieurs représentants d'une même ressource logique.
- Vous avez un objet sans état métier local qui coordonne une ressource unique du processus.
- Vous êtes tenté d'utiliser des variables globales mais souhaitez au moins encapsuler l'accès et l'initialisation.

2. Le symptôme central

À retenir : Singleton n'est pas "un objet pratique à accéder partout" ; c'est "une ressource qui doit, par contrainte, n'exister qu'une fois".

3. Scénario concret

Un registre de plugins chargé une seule fois doit être partagé par toutes les parties d'une petite application autonome.

4. À quoi ressemble le code avant ?

```
PluginRegistry a = new PluginRegistry();
PluginRegistry b = new PluginRegistry(); // deux registres divergents
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
PluginRegistry registry = PluginRegistry.getInstance();
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Vous voulez seulement "éviter de passer une dépendance partout" : préférez l'injection de dépendances.
- L'unicité n'est pas une contrainte du domaine mais une commodité.
- Vous voulez facilement isoler les tests ou exécuter plusieurs contextes en parallèle.

7. Question de diagnostic

L'unicité est-elle une contrainte réelle du système, ou seulement une commodité d'accès ?

8. Ne pas confondre

Une instance singleton peut être gérée par un conteneur DI sans implémenter le pattern classique `getInstance()`.

9. Règle mentale en une ligne

Singleton n'est pas "un objet pratique à accéder partout" ; c'est "une ressource qui doit, par contrainte, n'exister qu'une fois".

Patterns structurels

Le symptôme commun est un problème d'assemblage : interfaces incompatibles, hiérarchies qui explosent, objets arborescents, responsabilités empilées, sous-systèmes complexes ou accès indirect.

Dans cette partie, cherchez moins la forme UML parfaite que le signal concret indiquant que la responsabilité actuelle n'est plus placée au bon endroit.

06. Adapter

STRUCTURE

Essence — Rendre compatible une interface existante avec l'interface attendue par votre application.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez une classe ou API utile mais ses méthodes/paramètres ne correspondent pas à votre interface métier.
- Une librairie externe demande quatre données alors que votre port métier n'en expose que deux, ou inversement.
- Vous voyez du code de conversion de types et de renommage d'appels dispersé dans plusieurs services.
- Vous voulez remplacer un fournisseur externe sans contaminer le domaine avec son vocabulaire.
- Une ancienne API (legacy) doit continuer à fonctionner derrière une nouvelle interface.
- Les deux côtés font conceptuellement la même chose mais "ne parlent pas la même langue".

2. Le symptôme central

À retenir : Même intention, interfaces incompatibles → Adapter.

3. Scénario concret

Le domaine attend `PaymentPort.authorize(amount, currency)`, mais un SDK bancaire exige merchantId, token, cents et devise.

4. À quoi ressemble le code avant ?

```
bankSdk.authorize(config.merchantId(), tokenService.token(), amount.multiply(100), currency.code());
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
class BankPaymentAdapter implements PaymentPort {
    public Authorization authorize(Money amount, Currency currency) {
        return map(bankSdk.authorize(merchantId(), token(), toCents(amount), currency.code()));
    }
}
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Vous contrôlez les deux interfaces et pouvez simplement les faire évoluer proprement.
- Vous voulez ajouter des responsabilités sans changer l'interface : Decorator.
- Vous voulez simplifier une API complexe pour les clients : Facade.

7. Question de diagnostic

Les deux composants veulent-ils faire la même chose mais parlent-ils des interfaces différentes ?

8. Ne pas confondre

Adapter traduit ; Facade simplifie ; Bridge sépare deux dimensions variables ; Decorator enrichit.

9. Règle mentale en une ligne

Même intention, interfaces incompatibles → Adapter.

07. Bridge

STRUCTURE

Essence — Séparer une abstraction de son implémentation afin de faire évoluer indépendamment deux dimensions qui varient.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez deux axes de variation indépendants qui créent une explosion de sous-classes.
- Les noms de classes commencent à combiner deux dimensions : `RedCircle`, `BlueCircle`, `RedSquare`, `BlueSquare`.
- Ajouter une nouvelle abstraction multiplie le nombre d'implémentations à créer, ou inversement.
- Vous voulez pouvoir changer l'implémentation à l'exécution sans changer l'abstraction utilisée par le client.
- Une hiérarchie mélange "ce que l'objet fait" et "avec quelle technologie il le fait".
- Vous avez des plateformes ou backends multiples derrière plusieurs fonctionnalités de haut niveau.

2. Le symptôme central

À retenir : Deux dimensions qui varient indépendamment et se multiplient entre elles → séparez-les par un pont.

3. Scénario concret

Des télécommandes (basique, avancée) doivent piloter plusieurs appareils (TV, radio, projecteur). Faire une sous-classe par combinaison devient vite ingérable.

4. À quoi ressemble le code avant ?

```
class BasicTvRemote {}
class AdvancedTvRemote {}
class BasicRadioRemote {}
class AdvancedRadioRemote {}
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
class Remote {
    protected Device device;
}
class AdvancedRemote extends Remote { ... }

interface Device { void on(); void off(); }
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Il n'existe qu'un seul axe de variation.
- Une simple Strategy injectée suffit et vous n'avez pas réellement deux hiérarchies durables.
- La composition ajoute plus de complexité que la petite hiérarchie actuelle.

7. Question de diagnostic

Ai-je deux axes de variation indépendants qui multiplient les combinaisons de classes ?

8. Ne pas confondre

Bridge est structurel et durable ; Strategy remplace plutôt un algorithme ; Adapter réconcilie des interfaces déjà incompatibles.

9. Règle mentale en une ligne

Deux dimensions qui varient indépendamment et se multiplient entre elles → séparez-les par un pont.

08. Composite

STRUCTURE

Essence — Traiter de manière uniforme un objet simple et un groupe d'objets organisés en arbre.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Votre domaine est naturellement hiérarchique : dossiers/fichiers, menus/sous-menus, composants UI, organisation, catégories.
- Le client doit exécuter la même opération sur une feuille ou sur tout un sous-arbre.
- Vous avez des `instanceof` pour distinguer "élément simple" et "groupe".
- Une opération doit se propager récursivement aux enfants.
- Vous voulez pouvoir imbriquer des groupes à profondeur arbitraire.
- Le code client devrait pouvoir manipuler un composant sans savoir s'il contient lui-même des enfants.

2. Le symptôme central

À retenir : Si "un" et "un groupe de" doivent répondre au même message, pensez Composite.

3. Scénario concret

Calculer la taille d'un fichier ou d'un dossier entier avec la même méthode `size()`.

4. À quoi ressemble le code avant ?

```
if (node instanceof FileNode) return file.size();
else { long total=0; for (Node child: folder.children()) total += compute(child); }
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
interface Node { long size(); }
class FileNode implements Node { ... }
class Folder implements Node {
    List<Node> children;
    public long size(){ return children.stream().mapToLong(Node::size).sum(); }
}
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- La structure n'est pas arborescente.
- Feuilles et groupes ont des comportements totalement différents sans interface utile commune.
- Vous avez besoin de parcourir une collection linéaire : Iterator est probablement plus pertinent.

7. Question de diagnostic

Puis-je vouloir envoyer le même message à un élément et à un groupe d'éléments ?

8. Ne pas confondre

Composite représente l'arbre ; Iterator le parcourt ; Visitor peut ajouter des opérations sur cette structure.

9. Règle mentale en une ligne

Si “un” et “un groupe de” doivent répondre au même message, pensez Composite.

09. Decorator

STRUCTURE

Essence — Ajouter dynamiquement des responsabilités à un objet en l'enveloppant, sans multiplier les sous-classes.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez beaucoup de combinaisons optionnelles de comportements : compression + chiffrement + cache + logs.
- L'héritage produit des classes comme `LoggedCachedEncryptedService`.
- Les fonctionnalités doivent pouvoir être empilées dans différents ordres.
- Vous voulez enrichir seulement certaines instances, pas toute une classe.
- Le composant enrichi doit conserver la même interface que l'objet original.
- Chaque fonctionnalité transversale est simple mais leurs combinaisons explosent.

2. Le symptôme central

À retenir : Même interface + comportement ajouté autour de l'objet + combinaisons possibles → Decorator.

3. Scénario concret

Un flux de données peut être brut, compressé, chiffré, ou compressé puis chiffré selon le besoin.

4. À quoi ressemble le code avant ?

```
class CompressedEncryptedFileDataSource extends FileDataSource { ... }
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
DataSource source = new FileDataSource(path);
source = new CompressionDecorator(source);
source = new EncryptionDecorator(source);
source.write(data);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Vous voulez seulement remplacer un algorithme : Strategy.
- Vous voulez contrôler l'accès, le lazy loading ou la distance réseau : Proxy.
- L'ordre d'empilement devient incompréhensible et une pipeline explicite serait plus claire.

7. Question de diagnostic

Dois-je combiner librement plusieurs responsabilités optionnelles autour du même objet ?

8. Ne pas confondre

Decorator enrichit ; Proxy contrôle ; Adapter traduit ; Chain of Responsibility fait circuler une requête entre handlers.

9. Règle mentale en une ligne

Même interface + comportement ajouté autour de l'objet + combinaisons possibles → Decorator.

10. Facade

STRUCTURE

Essence — Offrir une interface simple et cohérente au-dessus d'un sous-système complexe.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Pour réaliser une opération métier simple, le client doit appeler cinq ou dix classes techniques dans le bon ordre.
- Plusieurs contrôleurs dupliquent la même orchestration de sous-système.
- Le code client connaît des détails internes qu'il ne devrait pas connaître.
- Une bibliothèque est puissante mais trop complexe pour les 80 % de cas courants.
- Vous souhaitez définir une frontière claire entre couches ou modules.
- Un changement interne provoque des modifications dans de nombreux clients.

2. Le symptôme central

À retenir : Si l'utilisateur doit connaître le mode d'emploi interne d'un sous-système pour une tâche simple, créez une façade.

3. Scénario concret

Pour convertir une vidéo, le client doit ouvrir le fichier, détecter codec, créer buffer, transcoder, mixer audio et sauvegarder. Une façade expose simplement `convert(input, MP4)`.

4. À quoi ressemble le code avant ?

```
VideoFile f = new VideoFile(path);
Codec c = CodecFactory.extract(f);
Buffer b = BitrateReader.read(f,c);
AudioMixer.fix(b);
File out = BitrateReader.convert(b,new Mpeg4Codec());
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
File out = videoConverter.convert(path, Format.MP4);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Le client a réellement besoin de toutes les capacités avancées du sous-système.
- Vous devez rendre deux interfaces compatibles : Adapter.
- Vous ne faites que cacher une classe sans simplifier une interaction significative.

7. Question de diagnostic

Le client doit-il connaître trop d'étapes internes juste pour accomplir une tâche simple ?

8. Ne pas confondre

Facade simplifie un ensemble ; Adapter transforme une interface ; Mediator réduit les dépendances entre pairs.

9. Règle mentale en une ligne

Si l'utilisateur doit connaître le mode d'emploi interne d'un sous-système pour une tâche simple, créez une façade.

11. Flyweight

STRUCTURE

Essence — Réduire la mémoire en partageant la partie intrinsèque et immuable de très nombreux objets similaires.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous créez des centaines de milliers ou millions d'objets très similaires.
- Le profil mémoire montre que les mêmes données lourdes sont dupliquées dans chaque instance.
- Une partie de l'état est identique et immuable pour une grande population d'objets.
- Vous pouvez séparer l'état intrinsèque partagé de l'état extrinsèque spécifique à chaque occurrence.
- Le coût mémoire est un vrai problème mesuré, pas une intuition.
- Les objets légers peuvent recevoir leur position/contexte au moment de l'opération.

2. Le symptôme central

À retenir : Beaucoup d'objets + énorme duplication d'état immuable → partagez cet état avec Flyweight.

3. Scénario concret

Dans un jeu, un million d'arbres ont des positions différentes mais seuls quelques types partagent texture, modèle 3D et couleur.

4. À quoi ressemble le code avant ?

```
new Tree(x, y, "oak", hugeTexture, mesh, color); // répété 1 000 000 fois
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
TreeType oak = factory.getTreeType("oak", texture, mesh, color);
new Tree(x, y, oak); // état lourd partagé
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Le volume d'objets est faible.
- L'état partagé est mutable et doit diverger par instance.
- La mesure montre que la mémoire n'est pas le goulot d'étranglement.

7. Question de diagnostic

La mémoire est-elle consommée par la duplication massive du même état immuable ?

8. Ne pas confondre

Prototype duplique ; Flyweight partage. Object Pool réutilise des ressources coûteuses mais n'est pas un pattern GoF.

9. Règle mentale en une ligne

Beaucoup d'objets + énorme duplication d'état immuable → partagez cet état avec Flyweight.

12. Proxy

STRUCTURE

Essence — Placer un représentant devant un objet afin de contrôler l'accès à celui-ci sans changer l'interface visible.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous voulez retarder la création d'un objet lourd jusqu'au premier usage (lazy loading).
- Vous devez vérifier des droits avant de déléguer à l'objet réel.
- L'objet réel est distant mais le client devrait l'utiliser comme un objet local.
- Vous voulez ajouter cache, métriques ou contrôle d'accès autour d'un service sans modifier le client.
- Vous avez besoin d'un substitut qui expose exactement la même interface que le sujet réel.
- Le problème principal n'est pas d'ajouter une fonctionnalité métier mais de contrôler quand/comment l'accès se produit.

2. Le symptôme central

À retenir : Même interface, mais un gardien/intermédiaire décide comment accéder à l'objet réel → Proxy.

3. Scénario concret

Une image haute résolution ne doit être chargée depuis le disque que lorsqu'elle doit réellement être affichée.

4. À quoi ressemble le code avant ?

```
Image img = new HighResolutionImage(path); // chargement coûteux immédiatement
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
Image img = new LazyImageProxy(path);  
// fichier réellement chargé lors de img.display()
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Vous voulez ajouter plusieurs responsabilités combinables : Decorator.
- Vous devez convertir une interface : Adapter.
- Vous contrôlez simplement un flux d'appels entre plusieurs handlers : Chain of Responsibility.

7. Question de diagnostic

Ai-je besoin d'un représentant qui contrôle l'accès à l'objet réel sans changer son interface ?

8. Ne pas confondre

Proxy contrôle l'accès ; Decorator enrichit le comportement ; Facade simplifie un sous-système.

9. Règle mentale en une ligne

Même interface, mais un gardien/intermédiaire décide comment accéder à l'objet réel → Proxy.

Patterns comportementaux

Le symptôme commun est un problème de collaboration : qui traite une requête, comment représenter une action, comment réagir à un événement, comment changer d'algorithme ou de comportement sans multiplier les conditions.

Dans cette partie, cherchez moins la forme UML parfaite que le signal concret indiquant que la responsabilité actuelle n'est plus placée au bon endroit.

13. Chain of Responsibility

COMPORTEMENT

Essence — Faire passer une requête à travers une chaîne de handlers, chacun pouvant traiter, enrichir, bloquer ou transmettre.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez une succession de validations ou filtres indépendants exécutés dans un ordre.
- Le code contient un gros bloc `if` avec plusieurs étapes de contrôle.
- Vous voulez ajouter, retirer ou réordonner des traitements sans modifier un contrôleur central.
- Chaque étape peut décider d'arrêter la chaîne ou de passer au suivant.
- Vous avez des middleware HTTP, filtres sécurité, règles d'approbation ou pipelines de validation.
- L'émetteur ne devrait pas connaître précisément quel handler prendra la décision finale.

2. Le symptôme central

À retenir : Une requête qui traverse des postes de contrôle indépendants → Chain of Responsibility.

3. Scénario concret

Une requête passe par authentification, autorisation, rate limiting puis contrôleur métier. Chaque middleware peut interrompre la suite.

4. À quoi ressemble le code avant ?

```
checkAuthentication(req);
checkAuthorization(req);
checkRateLimit(req);
validate(req);
controller.handle(req);
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
Handler chain = auth
    .setNext(authorize)
    .setNext(rateLimit)
    .setNext(validation);
chain.handle(request);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Toutes les étapes doivent impérativement s'exécuter et partagent beaucoup d'état : une pipeline explicite peut être plus claire.
- Vous cherchez un algorithme interchangeable unique : Strategy.
- Vous voulez encapsuler une action comme objet : Command.

7. Question de diagnostic

Une requête doit-elle traverser une succession de contrôles indépendants, éventuellement interrompue en chemin ?

8. Ne pas confondre

Chain distribue une requête ; Decorator empile des responsabilités sur un objet ; Pipeline est une architecture plus générale.

9. Règle mentale en une ligne

Une requête qui traverse des postes de contrôle indépendants → Chain of Responsibility.

14. Command

COMPORTEMENT

Essence — Encapsuler une action dans un objet afin de la stocker, planifier, journaliser, rejouer ou annuler.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous voulez mettre des actions dans une file d'attente ou les exécuter plus tard.
- Vous avez besoin d'undo/redo.
- Une interface graphique possède de nombreux boutons dont les actions doivent être interchangeables.
- Vous voulez journaliser ou rejouer précisément des opérations utilisateur.
- L'émetteur d'une action ne devrait pas connaître le receveur concret ni la logique de l'action.
- Vous avez des jobs, tâches, transactions applicatives ou opérations planifiées représentant des verbes métier.

2. Le symptôme central

À retenir : Si une action doit devenir une "chose" manipulable comme une donnée, faites-en un Command.

3. Scénario concret

Un éditeur de texte doit exécuter `Copy`, `Paste`, `Delete`, garder l'historique et annuler les opérations réversibles.

4. À quoi ressemble le code avant ?

```
if (button.equals("copy")) editor.copy();
else if (button.equals("paste")) editor.paste();
else if (button.equals("delete")) editor.deleteSelection();
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
Command command = new DeleteCommand(editor);
command.execute();
history.push(command);
// plus tard : command.undo();
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- L'action est triviale, immédiate et ne sera jamais stockée/annulée/rejouée.
- Vous cherchez seulement à remplacer une formule ou un algorithme : Strategy.
- Vous avez besoin d'une série d'étapes fixes dans une méthode : Template Method.

7. Question de diagnostic

Ai-je besoin de manipuler une action comme une donnée : la stocker, l'annuler, la rejouer ou la planifier ?

8. Ne pas confondre

Command encapsule une action ; Strategy encapsule un algorithme ; Memento capture un état pour l'annulation.

9. Règle mentale en une ligne

Si une action doit devenir une "chose" manipulable comme une donnée, faites-en un Command.

15. Interpreter

COMPORTEMENT

Essence — Représenter une petite grammaire sous forme d'objets et interpréter des expressions de ce langage.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez un mini-langage ou une syntaxe répétitive à évaluer : filtres, règles, expressions simples.
- Le code parse des expressions puis contient de nombreux `if` sur les types d'opérateurs.
- La grammaire est suffisamment simple et stable pour être représentée par un arbre d'expressions.
- Vous voulez composer des règles : AND, OR, NOT, comparaison, variables, littéraux.
- Chaque élément syntaxique peut naturellement devenir une classe ou un nœud.
- Vous avez besoin d'évaluer plusieurs fois des expressions déjà parsées.

2. Le symptôme central

À retenir : Petite grammaire répétée + arbre d'expressions + évaluation → Interpreter.

3. Scénario concret

Un moteur de recherche interne accepte `country = FR AND age > 18` et doit évaluer la règle sur de nombreux objets.

4. À quoi ressemble le code avant ?

```
String[] parts = rule.split(" ");
// nombreux if sur AND, OR, >, =, parenthèses...
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
Expression rule = new And(
    new Equals(new Field("country"), new Literal("FR")),
    new GreaterThan(new Field("age"), new Literal(18))
);
boolean ok = rule.interpret(context);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- La grammaire est complexe : utilisez un vrai parser/ANTLR ou un moteur de règles.
- La syntaxe change fréquemment et devient un langage généraliste.
- Vous n'avez qu'une poignée de conditions codées directement et lisiblement.

7. Question de diagnostic

Suis-je en train de réimplémenter encore et encore l'évaluation d'un petit langage ou d'expressions ?

8. Ne pas confondre

Composite sert souvent à représenter l'arbre syntaxique ; Visitor peut réaliser des opérations sur cet arbre.

9. Règle mentale en une ligne

Petite grammaire répétée + arbre d'expressions + évaluation → Interpreter.

16. Iterator

COMPORTEMENT

Essence — Parcourir une collection sans exposer sa représentation interne.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Le client doit parcourir une structure mais ne devrait pas savoir si elle est liste, arbre, graphe ou stockage paginé.
- Vous dupliquez la logique d'indexation/parcours à plusieurs endroits.
- Plusieurs stratégies de parcours sont nécessaires : profondeur, largeur, ordre filtré.
- Vous voulez permettre plusieurs parcours indépendants de la même collection.
- Une collection a une structure interne compliquée mais le client veut juste demander "suivant".
- Vous voulez fournir une boucle uniforme au-dessus de structures différentes.

2. Le symptôme central

À retenir : Le client veut parcourir sans connaître la structure → Iterator.

3. Scénario concret

Une structure d'organisation est un arbre, mais les clients doivent pouvoir parcourir les employés sans connaître les nœuds internes.

4. À quoi ressemble le code avant ?

```
for (Department d : company.departments) {
    for (Team t : d.teams) {
        for (Employee e : t.employees) { ... }
    }
}
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
Iterator<Employee> it = company.employeeIterator();
while (it.hasNext()) {
    Employee e = it.next();
}
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Votre langage fournit déjà exactement l'itérateur nécessaire et vous n'ajoutez rien.
- Vous avez besoin d'appliquer une nouvelle opération complexe à chaque type de nœud : Visitor peut être le vrai sujet.
- Vous cherchez à traiter feuille et groupe pareil : Composite.

7. Question de diagnostic

Le client devrait-il parcourir la structure sans connaître son organisation interne ?

8. Ne pas confondre

Composite structure l'arbre ; Iterator le parcourt ; Visitor opère selon le type rencontré.

9. Règle mentale en une ligne

Le client veut parcourir sans connaître la structure → Iterator.

17. Mediator

COMPORTEMENT

Essence — Centraliser les interactions entre plusieurs objets afin qu'ils n'aient pas à se connaître directement.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Plusieurs composants se référencent mutuellement et le graphe de dépendances devient illisible.
- Une modification dans un composant nécessite de modifier plusieurs autres composants qui l'appellent directement.
- Vous avez une UI avec champs, boutons et listes qui se déclenchent mutuellement.
- Les interactions sont nombreuses mais chacun des composants pris isolément est simple.
- Vous voulez réutiliser un composant sans embarquer toutes ses dépendances croisées.
- Le code contient beaucoup d'appels du type ``a.notifyB()``, ``b.updateC()``, ``c.resetA()``.

2. Le symptôme central

À retenir : Quand tout le monde parle à tout le monde, introduisez un organisateur de conversation.

3. Scénario concret

Dans une boîte de dialogue, cocher “authentification entreprise” active certains champs, désactive le mot de passe et change les boutons disponibles.

4. À quoi ressemble le code avant ?

```
checkbox.setForm(form);
checkbox.setPasswordField(password);
password.setCheckbox(checkbox);
button.setCheckbox(checkbox);
button.setPassword(password);
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
checkbox.setMediator(dialog);
password.setMediator(dialog);
button.setMediator(dialog);
// dialog coordonne les réactions
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Il n'y a que deux objets avec une collaboration simple et stable.
- Le médiateur devient un “God Object” contenant toute la logique métier.
- Les composants sont surtout intéressés par des événements diffusés à plusieurs abonnés : Observer peut être plus naturel.

7. Question de diagnostic

Le vrai problème est-il que trop de composants se connaissent et se déclenchent mutuellement ?

8. Ne pas confondre

Mediator organise des pairs ; Facade simplifie l'accès externe à un sous-système ; Observer diffuse des événements.

9. Règle mentale en une ligne

Quand tout le monde parle à tout le monde, introduisez un organisateur de conversation.

18. Memento

COMPORTEMENT

Essence — Capturer l'état interne d'un objet afin de pouvoir le restaurer plus tard sans exposer cet état au reste du système.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez besoin de rollback, snapshots ou undo basé sur l'état.
- Pour annuler, le code extérieur doit lire et recopier beaucoup de champs privés d'un objet.
- Vous voulez sauvegarder un état sans violer l'encapsulation.
- Un objet sait mieux que quiconque quelles données sont nécessaires pour restaurer un état cohérent.
- Vous devez pouvoir revenir à plusieurs points antérieurs.
- La reconstruction inverse d'une action est plus difficile que restaurer un snapshot.

2. Le symptôme central

À retenir : Besoin de revenir à "comme c'était avant" sans exposer les entrailles de l'objet → Memento.

3. Scénario concret

Un éditeur doit revenir à l'état du document avant une transformation complexe dont l'inverse serait difficile à calculer.

4. À quoi ressemble le code avant ?

```
backupText = editor.getText();
backupCursor = editor.getCursor();
backupSelection = editor.getSelection();
// le caretaker connaît les détails internes
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
EditorMemento snapshot = editor.save();
// ... modifications ...
editor.restore(snapshot);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- L'état est énorme et les snapshots fréquents seraient trop coûteux.
- Une opération a un inverse naturel et léger : Command + undo peut suffire.
- La persistance durable versionnée nécessite plutôt un mécanisme de stockage métier explicite.

7. Question de diagnostic

Dois-je restaurer un état passé sans exposer tous les détails internes de l'objet ?

8. Ne pas confondre

Memento capture un état ; Command capture une action ; Event Sourcing reconstruit l'état à partir d'événements et n'est pas un pattern GoF.

9. Règle mentale en une ligne

Besoin de revenir à “comme c’était avant” sans exposer les entrailles de l’objet → Memento.

19. Observer

COMPORTEMENT

Essence — Notifier automatiquement plusieurs abonnés lorsqu'un objet change ou publie un événement.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Lorsqu'un événement se produit, plusieurs composants indépendants doivent réagir.
- Le producteur ne devrait pas connaître la liste concrète des consommateurs.
- Vous avez du code qui appelle manuellement `emailService`, `statsService`, `ui`, `audit` après chaque changement.
- Le nombre d'abonnés peut varier à l'exécution.
- Vous voulez réduire le couplage entre source d'événements et réactions.
- Le modèle doit pouvoir notifier plusieurs vues ou projections.

2. Le symptôme central

À retenir : Un événement, plusieurs réactions indépendantes, producteur découplé → Observer.

3. Scénario concret

Après création d'une commande, l'application met à jour l'UI, les statistiques et une notification, sans que `Order` connaisse tous ces services.

4. À quoi ressemble le code avant ?

```
order.save();
ui.refresh(order);
stats.update(order);
notification.send(order);
audit.record(order);
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
orderEvents.subscribe(uiListener);
orderEvents.subscribe(statsListener);
orderEvents.subscribe(notificationListener);
orderEvents.publish(new OrderCreated(order));
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- L'ordre et la garantie transactionnelle entre réactions sont critiques et doivent être orchestrés explicitement.
- Vous avez un unique collaborateur obligatoire : une dépendance directe est plus claire.
- Les abonnements non maîtrisés risquent de provoquer boucles, fuites mémoire ou effets invisibles.

7. Question de diagnostic

Un même événement doit-il provoquer plusieurs réactions indépendantes et variables ?

8. Ne pas confondre

Observer diffuse à plusieurs abonnés ; Mediator coordonne des interactions ; Pub/Sub distribué est une généralisation architecturale.

9. Règle mentale en une ligne

Un événement, plusieurs réactions indépendantes, producteur découplé → Observer.

20. State

COMPORTEMENT

Essence — Faire varier le comportement d'un objet selon son état interne en déléguant ce comportement à des objets d'état.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Une méthode contient beaucoup de `if (state == ...)` ou `switch(state)`.
- Les mêmes tests d'état sont répétés dans plusieurs méthodes.
- Les transitions entre états ont des règles propres et deviennent difficiles à suivre.
- Ajouter un nouvel état oblige à modifier de nombreuses méthodes.
- Le même événement doit produire un comportement différent selon l'état courant.
- Votre objet ressemble à une machine à états : commande NEW/PAID/SHIPPED/CANCELLED, connexion, workflow de document.

2. Le symptôme central

À retenir : Les `if` dépendent de "dans quel état suis-je ?" et se répètent partout → State.

3. Scénario concret

Une commande peut être payée, expédiée ou annulée selon son état. `cancel()` n'a pas le même sens avant paiement et après expédition.

4. À quoi ressemble le code avant ?

```
switch (status) {
  case NEW -> ...;
  case PAID -> ...;
  case SHIPPED -> throw ...;
  case CANCELLED -> ...;
}
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
interface OrderState { void cancel(Order order); void pay(Order order); }
class PaidState implements OrderState { ... }
class ShippedState implements OrderState { ... }
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Il n'y a que deux états avec une condition évidente et stable.
- Vous changez seulement une formule/algorithmes sans notion de transition : Strategy.
- Les transitions sont mieux décrites par un moteur de workflow externe.

7. Question de diagnostic

Mes conditions portent-elles sans cesse sur l'état courant du même objet ?

8. Ne pas confondre

State et Strategy utilisent la composition ; State change souvent de lui-même au fil des transitions, Strategy est choisie comme politique.

9. Règle mentale en une ligne

Les `if` dépendent de “dans quel état suis-je ?” et se répètent partout → State.

21. Strategy

COMPORTEMENT

Essence — Encapsuler plusieurs algorithmes interchangeables derrière une même interface et choisir celui à utiliser selon le contexte.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Une méthode contient un `if` ou `switch` pour choisir une formule ou un algorithme.
- Vous avez plusieurs façons de réaliser la même tâche : remise, tri, routage, compression, tarification.
- Les variantes changent indépendamment du reste de la classe.
- Vous voulez tester chaque algorithme isolément.
- Une classe grossit parce qu'elle contient toutes les variantes dans des méthodes privées.
- Le choix de l'algorithme dépend de la configuration, du client ou du contexte.

2. Le symptôme central

À retenir : Même objectif, plusieurs manières interchangeables de l'atteindre → Strategy.

3. Scénario concret

Un calcul de frais de livraison peut utiliser une stratégie standard, express ou retrait magasin.

4. À quoi ressemble le code avant ?

```
if (mode == STANDARD) cost = standard(order);
else if (mode == EXPRESS) cost = express(order);
else if (mode == PICKUP) cost = 0;
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
ShippingStrategy strategy = strategyResolver.forMode(mode);
Money cost = strategy.compute(order);
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Il n'existe qu'un seul algorithme réaliste.
- Les variantes correspondent en réalité à des états avec transitions : State.
- L'algorithme a un squelette invariant avec quelques étapes redéfinies : Template Method.

7. Question de diagnostic

Mes conditions choisissent-elles surtout entre plusieurs algorithmes pour le même objectif ?

8. Ne pas confondre

Strategy = algorithme interchangeable ; State = comportement selon état ; Template Method = squelette commun par héritage.

9. Règle mentale en une ligne

Même objectif, plusieurs manières interchangeables de l'atteindre → Strategy.

22. Template Method

COMPORTEMENT

Essence — Définir le squelette d'un algorithme dans une classe mère et laisser certaines étapes être spécialisées par les sous-classes.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Plusieurs classes suivent exactement la même séquence mais diffèrent sur quelques étapes.
- Vous copiez-collez un workflow entier pour modifier seulement une opération.
- L'ordre des étapes doit être imposé et commun à toutes les variantes.
- Certaines étapes sont obligatoires, d'autres sont des hooks optionnels.
- Vous voulez factoriser l'invariant d'un processus dans une classe de base.
- Le choix des variations est naturellement lié à une sous-classe et reste stable pendant la vie de l'objet.

2. Le symptôme central

À retenir : Même recette, mêmes étapes dans le même ordre, quelques étapes variables → Template Method.

3. Scénario concret

Importer CSV et JSON suit : ouvrir → lire → parser → valider → sauvegarder → fermer. Seul le parsing diffère fortement.

4. À quoi ressemble le code avant ?

```
class CsvImporter { import(){ open(); read(); parseCsv(); validate(); save(); close(); }}
class JsonImporter { import(){ open(); read(); parseJson(); validate(); save(); close(); }}
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
abstract class Importer {
    public final void importData(){ open(); read(); parse(); validate(); save(); close(); }
    protected abstract void parse();
}
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- Vous devez changer l'algorithme à l'exécution : Strategy est plus flexible.
- L'héritage crée un couplage gênant ou une hiérarchie artificielle.
- Les étapes ne sont pas vraiment communes ou leur ordre varie beaucoup.

7. Question de diagnostic

Plusieurs classes répètent-elles la même recette en ne variant que certaines étapes ?

8. Ne pas confondre

Template Method utilise l'héritage ; Strategy utilise la composition ; Factory Method est souvent une étape spécialisée dans un template.

9. Règle mentale en une ligne

Même recette, mêmes étapes dans le même ordre, quelques étapes variables → **Template Method**.

23. Visitor

COMPORTEMENT

Essence — Ajouter de nouvelles opérations à une structure d'objets hétérogènes sans modifier chaque classe de cette structure.

1. Symptômes situationnels

Ce pattern mérite votre attention lorsque plusieurs des signaux suivants apparaissent ensemble :

- Vous avez une structure de classes stable mais vous ajoutez souvent de nouvelles opérations transversales dessus.
- Une opération dépend fortement du type concret de chaque élément.
- Vous accumulez des `instanceof` pour traiter différemment Circle, Rectangle, Group, etc.
- Vous voulez exporter, valider, calculer, sérialiser ou analyser les mêmes nœuds de plusieurs façons.
- Ajouter une nouvelle opération devrait être plus fréquent qu'ajouter un nouveau type de nœud.
- Vous avez un arbre Composite sur lequel de nombreuses opérations distinctes doivent s'appliquer.

2. Le symptôme central

À retenir : Structure de types stable + nouvelles opérations fréquentes + `instanceof` répétés → Visitor.

3. Scénario concret

Un modèle géométrique contient plusieurs types de formes. On ajoute successivement export XML, calcul de statistiques et validation, sans modifier toutes les formes à chaque fois.

4. À quoi ressemble le code avant ?

```
if (shape instanceof Circle c) exportCircle(c);
else if (shape instanceof Rectangle r) exportRectangle(r);
else if (shape instanceof Group g) exportGroup(g);
```

Ce qui doit vous alerter : la responsabilité problématique est visible directement dans le code client : choix, connaissance concrète, duplication, branchement ou orchestration qui ne devrait probablement pas lui appartenir.

5. Après introduction du pattern

```
interface ShapeVisitor {
    void visit(Circle c);
    void visit(Rectangle r);
    void visit(Group g);
}
shape.accept(new exportVisitor());
```

Ce qui a changé : la variation est déplacée derrière une abstraction plus adaptée. Le code client exprime davantage son intention et connaît moins les détails de réalisation.

6. Quand ne PAS utiliser ce pattern

- De nouveaux types de nœuds sont ajoutés très fréquemment : chaque nouveau type oblige à modifier tous les visiteurs.
- L'opération appartient naturellement à la classe elle-même.
- Le double dispatch rend le design plus difficile à comprendre que le problème initial.

7. Question de diagnostic

La structure de types est-elle stable alors que j'ajoute régulièrement de nouvelles opérations qui dépendent du type concret ?

8. Ne pas confondre

Visitor ajoute des opérations ; Composite représente souvent la structure parcourue ; Iterator fournit le parcours sans discrimination par type.

9. Règle mentale en une ligne

Structure de types stable + nouvelles opérations fréquentes + ``instanceof`` répétés → Visitor.

Carte de diagnostic rapide : partir du symptôme

Cette section sert de raccourci. Partez de ce que vous voyez réellement dans le code puis regardez les patterns les plus plausibles. Ce n'est pas une règle automatique : c'est un filtre pour orienter votre réflexion.

Symptôme observé	Patterns à examiner	Question clé
Beaucoup de `new` choisis par `if/switch`	Factory Method / Abstract Factory	Le type concret ou la famille de types varie.
Constructeur énorme, beaucoup d'options	Builder	La difficulté est la construction lisible et progressive.
"Le même objet mais avec quelques différences"	Prototype	On part d'un exemplaire existant.
Ressource réellement unique dans le processus	Singleton	L'unicité est une contrainte, pas juste une commodité.
API utile mais interface incompatible	Adapter	Il faut traduire une interface vers une autre.
Explosion de classes par combinaison de 2 dimensions	Bridge	Deux axes varient indépendamment.
Arbre : élément et groupe doivent se manipuler pareil	Composite	Structure partie-tout récursive.
Combinaisons de fonctionnalités autour du même objet	Decorator	Responsabilités optionnelles empilables.
Sous-système compliqué à utiliser	Facade	Il faut une entrée simple et stable.
Millions d'objets avec données identiques dupliquées	Flyweight	Partager l'état intrinsèque économise la mémoire.
Lazy loading, sécurité, distant, cache devant un service	Proxy	Contrôler l'accès à l'objet réel.
Suite de contrôles indépendants	Chain of Responsibility	La requête circule de handler en handler.
Action à stocker/annuler/rejouer	Command	L'action devient un objet.
Mini-langage / expressions à évaluer	Interpreter	La grammaire devient un arbre d'expressions.
Parcours sans exposer la structure	Iterator	Séparer parcours et représentation.

Symptôme observé	Patterns à examiner	Question clé
interne		
Trop d'objets se connaissent mutuellement	Mediator	Centraliser les interactions.
Snapshot / retour à un état antérieur	Memento	Capturer puis restaurer l'état.
Un événement déclenche plusieurs réactions	Observer	Publier à des abonnés découplés.
Beaucoup de <code>if(state == ...)</code>	State	Le comportement dépend de l'état courant.
Beaucoup de <code>if(typeAlgo == ...)</code>	Strategy	Le comportement dépend d'un algorithme interchangeable.
Même recette, quelques étapes variables	Template Method	Squelette invariant, étapes spécialisées.
Beaucoup de <code>instanceof</code> pour de nouvelles opérations	Visitor	Opérations variables sur une structure de types stable.

Distinguer les patterns que l'on confond souvent

Factory Method vs Strategy

Factory Method choisit quel objet créer. Strategy choisit quel algorithme exécuter. Le premier traite la création ; le second le comportement.

State vs Strategy

Ils se ressemblent structurellement. Strategy représente une politique choisie ; State représente une situation interne qui évolue et entraîne des transitions.

Adapter vs Facade

Adapter rend compatible une interface existante. Facade propose une interface plus simple à un ensemble complexe.

Decorator vs Proxy

Les deux enveloppent un objet avec la même interface. Decorator ajoute des responsabilités ; Proxy contrôle l'accès au sujet réel.

Decorator vs Chain of Responsibility

Decorator applique typiquement toutes les couches empilées. Dans une chaîne, un handler peut traiter ou interrompre la propagation.

Bridge vs Strategy

Bridge sépare durablement deux dimensions d'un modèle. Strategy isole surtout une famille d'algorithmes interchangeables.

Command vs Memento

Command mémorise l'action et peut parfois fournir son inverse. Memento mémorise l'état à restaurer.

Composite vs Iterator vs Visitor

Composite représente la structure, Iterator fournit le parcours, Visitor ajoute des opérations dépendantes des types rencontrés.

Observer vs Mediator

Observer diffuse un événement à des abonnés. Mediator coordonne les interactions entre plusieurs collègues et peut contenir les règles d'orchestration.

Template Method vs Strategy

Template Method varie par héritage à l'intérieur d'un squelette imposé. Strategy varie par composition et peut être remplacée dynamiquement.

Checklist finale avant d'introduire un pattern

- Je peux nommer le problème concret que le pattern doit résoudre.
- Le problème existe réellement dans le code actuel ou dans une évolution proche, pas seulement dans mon imagination.
- Le pattern déplace une responsabilité vers un endroit plus cohérent.
- Le prochain changement attendu devient plus local et plus simple.
- Le nombre de classes supplémentaires est justifié par la réduction du couplage ou de la duplication.
- Un développeur qui découvre le code pourra expliquer pourquoi cette abstraction existe.
- Je sais également dans quelles conditions je supprimerais ce pattern si le besoin disparaissait.

Un bon pattern doit rendre une variation importante plus facile à absorber. S'il ne fait qu'augmenter le nombre de classes, ce n'est pas une amélioration de conception.

La règle générale

REPÉREZ CE QUI VARIE, PUIS ISOLEZ CETTE VARIATION.

C'est le fil conducteur derrière une grande partie des patterns GoF : le type créé varie, l'algorithme varie, l'état varie, la représentation varie, l'accès varie, la famille de composants varie ou la manière de collaborer varie. Le pattern pertinent est celui qui place cette variation derrière la bonne frontière.